

TP 2 - FastAPI

TP 2 — Compléter son CRUD (Update & Delete)

Durée indicative : 2h30. Vous avancez à votre rythme, ce n'est pas un chrono strict.

Avant de commencer

Ce TP part de l'API que vous avez commencée au TP 1 (avec au moins du GET et du POST qui fonctionnent). Si elle n'est pas terminée, l'étape 1 sert justement à la compléter avant d'avancer.

Rappel express

Un CRUD, ce sont les 4 opérations de base qu'on retrouve sur presque toutes les ressources d'une API :

Lettre	Opération	Verbe HTTP	Exemple
Create	Créer	POST	POST /books
Read	Lire	GET	GET /books/3
Update	Modifier	PATCH	PATCH /books/3
Delete	Supprimer	DELETE	DELETE /books/3

Vous avez déjà fait le C et le R au TP 1. Aujourd'hui : le U et le D.

Étape 1 — Reprendre son API (15 min)

1. Relancez votre serveur :

```
uvicorn main:app --reload
```

2. Vérifiez que vos routes `GET` et `POST` fonctionnent toujours (testez via `http://127.0.0.1:8000/docs`).
3. Si votre TP 1 n'est pas terminé, complétez-le en priorité avant de continuer : vous devez avoir 2 ressources, avec une route GET (liste), une route GET (détail avec paramètre), et une route POST, sur chacune.
4. Repérez dans votre code où sont stockées vos listes de données (`books = [...]` par exemple), vous allez les manipuler toute la séance.

Étape 2 — Ajouter la modification, Update (40 min)

Objectif : pouvoir modifier un élément existant sans toucher aux autres champs.

1. Ajoutez une route `PATCH` sur votre première ressource :

```
@app.patch("/books/{book_id}")
def update_book(book_id: int, changes: dict):
    for book in books:
        if book["id"] == book_id:
            book.update(changes)
            return book
    return {"error": "non trouvé"}
```

Comment ça marche :

- `for book in books` : on parcourt la liste pour retrouver le bon élément
 - `book.update(changes)` : une méthode Python qui fusionne les nouveaux champs reçus dans l'objet existant — seuls les champs que vous envoyez sont modifiés, le reste reste inchangé
 - Si aucun livre ne correspond à l'id demandé, on renvoie un message d'erreur
2. Testez depuis `/docs` : modifiez un seul champ (par exemple juste le titre) et vérifiez que les autres champs (auteur, etc.) n'ont pas bougé.

3. Testez aussi avec un id qui n'existe pas : vous devez voir le message d'erreur.
4. Faites la même chose sur votre deuxième ressource.

⚠ Ne confondez pas PATCH et PUT : PATCH modifie seulement les champs fournis (ce qu'on utilise ici), PUT remplacerait l'objet entier.

Étape 3 — Ajouter la suppression, Delete (30 min)

Objectif : pouvoir supprimer un élément de la liste.

1. Ajoutez une route `DELETE` sur votre première ressource :

```
@app.delete("/books/{book_id}")
def delete_book(book_id: int):
    global books
    books = [b for b in books if b["id"] != book_id]
    return {"message": "supprimé"}
```

Comment ça marche :

- `[b for b in books if b["id"] != book_id]` : une "list comprehension", une syntaxe Python qui reconstruit la liste en gardant tous les éléments SAUF celui qu'on veut supprimer
 - `global books` : nécessaire ici car on réassigne complètement la variable `books` (on ne fait pas que la modifier)
2. Vérifiez avec un `GET` que l'élément supprimé n'apparaît plus dans la liste.
 3. Faites la même chose sur votre deuxième ressource.

Étape 4 — Un CRUD complet et propre (45 min)

Objectif : consolider, pas ajouter de nouvelles fonctionnalités.

1. Vérifiez que vos 2 ressources ont bien les 4 opérations CRUD complètes (Create, Read en liste, Read en détail, Update, Delete).
2. Relisez vos noms de fonctions et de routes : sont-ils clairs ? Cohérents entre vos deux ressources (par exemple `get_books` / `get_movies` , pas `get_books` / `movieDetail`) ?
3. Regroupez visuellement les routes par ressource dans votre fichier `main.py` : toutes les routes concernant les livres ensemble, puis toutes celles concernant votre deuxième ressource.

Étape 5 — Vérifier et bonus (20 min)

1. Testez l'intégralité de votre CRUD via `/docs` , dans l'ordre logique : Create → Read → Update → Delete.
2. **Bonus** : que se passe-t-il si vous faites un `DELETE` deux fois de suite sur le même id ? Est-ce que le comportement vous semble correct ?
3. **Bonus** : ajoutez une route qui renvoie simplement le nombre total d'éléments d'une ressource (ex : `GET /books/count`).

Contrainte minimale à respecter

À la fin du TP, votre API doit avoir, sur au moins une ressource :

- ✔ **Create** — une route POST fonctionnelle
- ✔ **Read** — une route GET liste et une route GET détail
- ✔ **Update** — une route PATCH qui modifie sans écraser les autres champs
- ✔ **Delete** — une route DELETE qui retire bien l'élément
- ✔ Toutes ces routes testées et fonctionnelles via `/docs`

En cas de blocage

Appelez-moi dès que vous êtes bloqué plus de 5 minutes sur quelque chose, pas la peine de rester coincé seul dans votre coin.